# 5 Grafos dirigidos

Formalmente, um ***grafo dirigido*** é um par $(V, E)$ onde $V$ é um conjunto finito (vértices) e $E \subseteq V \times V$ é o conjunto das arestas, ou ***arcos*** do grafo dirigido. Quando vale a restrição $(v, v) \notin E$, para todo $v \in V$, chamamos de ***grafo dirigido simples*** (podem existir $(u, v)$ e $(v, u)$ simultaneamente).

Dizemos que a aresta, ou ***arco***, $(v, u) \in E$ ***sai de*** $v$ e ***chega em*** $u$. Também, definimos os subconjuntos $N^+(v)$, $N^-(v) \subseteq V$, de *vizinhos* de cada $v \in V$

$$N^+(v) \coloneqq \{u \in V : (v, u) \in E\} \quad \text{e} \quad N^-(v) \coloneqq \{u \in V : (u, v) \in E\}.$$

Definimos, para cada $v \in V$, o ***grau de saída***, $\mathrm{d}^+(v)$, e o ***grau de entrada***, $\mathrm{d}^-(v)$, respectivamente, por

$$\mathrm{d}^+(v) \coloneqq |N^+(v)| \quad \text{e} \quad \mathrm{d}^-(v) \coloneqq |N^-(v)|$$

e é fácil de verificar que vale

$$\sum_{v \in V} \mathrm{d}^+(v) = \sum_{v \in V} \mathrm{d}^-(v) = |E|,$$

pois cada arco $(u, v)$ é contado uma vez em $\mathrm{d}^+(u)$ e uma vez em $\mathrm{d}^-(v)$.

O ***grafo subjacente*** ao grafo dirigido $G = (V, E)$ é o grafo $SG$ dado por

$$SG = \left(V, \bigcup_{\substack{(u,v) \in E \\ u \neq v}} \{\{u, v\}\}\right)$$

## 5.1 Representação computacional e percurso

De modo análogo ao caso não dirigido, assumiremos que nos problemas computacionais os vértices de $G$ são $\{1, 2, \ldots, n\}$ (caso contrário construímos um isomorfismo).

Como antes, um grafo dirigido $(V, E)$ pode ser representado por uma matriz $A$ quadrada $|V| \times |V|$ definida por

$$A(i, j) = \begin{cases} 1, & \text{se } (i, j) \in E \\ 0, & \text{caso contrário,} \end{cases}$$

ou seja, é uma matriz com entradas 0 e 1 dada por $a_{i,j} = 1$ se e só se $(i, j)$ é arco. Notemos que a matriz agora pode não ser simétrica. Para o grafo $G$ representado na Figura 5.1, a matriz de adjacência é

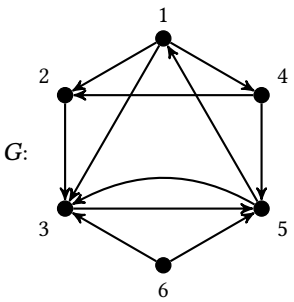



**Figura 5.1:** Grafo dirigido representado na matriz (5.2) e na lista de adjacências da Figura 5.2.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}. \tag{5.2}$$

Também podemos representar um grafo por sua ***lista de adjacência dirigida***, dada por um vetor $N^+[\,]$ de listas ligadas. A lista $N^+[i]$ contém os vizinhos $N^+(i)$ do vértice $i$ e cada nó dessa lista é composto por uma variável que armazena um vértice e um ponteiro que aponta para o próximo nó da lista.

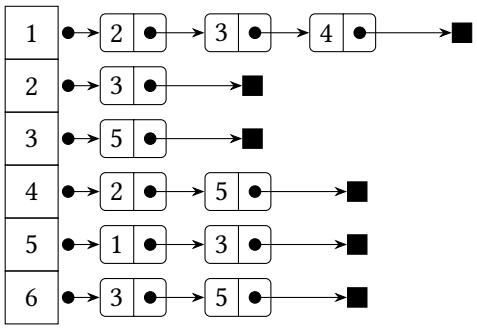



**Figura 5.2:** Lista de adjacência de um grafo dirigido com 6 vértices.

### 5.1.1 Busca em profundidade

Uma busca em profundidade rotulada, algoritmo 3 na página 32 reproduzido parcialmente na margem ao lado, executada num grafo dirigido resulta numa classificação dos arcos em quatro tipos, ao invés dos dois tipos (árvore/retorno) que ocorrem no caso não dirigido.

```
1 cont ← cont + 1
2 chega[w] ← cont
3 para cada u ∈ N⁺(w) faça
4     se chega[u] = 0 então
5         p[u] ← w
6         BP(G, u)
7 cont ← cont + 1
8 sai[w] ← cont.
```

Considere uma execução completa da busca em profundidade rotulada, que recomeça em um vértice ainda não visitado até visitar todos. Seja $p[v]$ o pai de $v$, com $p[v] = \texttt{nil}$ se $v$ é raiz de uma árvore da busca, e defina $p^{(0)}[v] = v$ e $p^{(t)}[v] = p[p^{(t-1)}[v]]$ enquanto o resultado está definido. Dizemos que $u$ é ***ancestral próprio*** de $v$ (e $v$ ***descendente próprio*** de $u$) se $p^{(t)}[v] = u$ para algum $t \geq 1$. Se $p[u] = v$, dizemos que $u$ é ***filho*** de $v$ e que $v$ é ***pai*** de $u$.

**Proposição 5.1: Classificação dos arcos**

Após uma busca em profundidade rotulada, todo arco $(u, v) \in E$ satisfaz exatamente uma das condições

1. *arco pai–filho*: $p[v] = u$;
2. *retorno ascendente*: $v$ é ancestral próprio de $u$, caso em que $[\text{chega}[u], \text{sai}[u]] \subset [\text{chega}[v], \text{sai}[v]]$;
3. *retorno descendente*: $v$ é descendente próprio de $u$ e $p[v] \neq u$, caso em que $[\text{chega}[v], \text{sai}[v]] \subset [\text{chega}[u], \text{sai}[u]]$;

4. *arco transversal*: chega[$v$] < sai[$v$] < chega[$u$] < sai[$u$].

(A prova usa que, para $x \neq y$, os intervalos [chega, sai] são disjuntos ou um contém o outro, e que chega[$u$] < chega[$v$] implica $v$ descendente de $u$ se $v$ é branco quando $u$ é explorado; veja Cormen et al. (2012).)

A figura abaixo esquematiza uma busca em profundidade rotulada no grafo dirigido do exemplo 5.2.

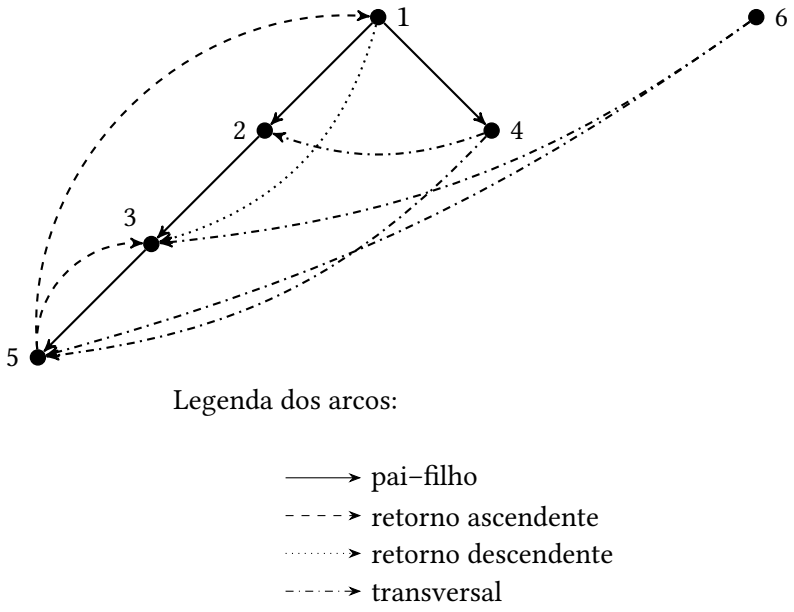



**Figura 5.3:** Busca em profundidade rotulada.

*Exercício* 83. A Figura 5.5 é o resultado de uma busca em profundidade rotulada no grafo dirigido da Figura 5.4.

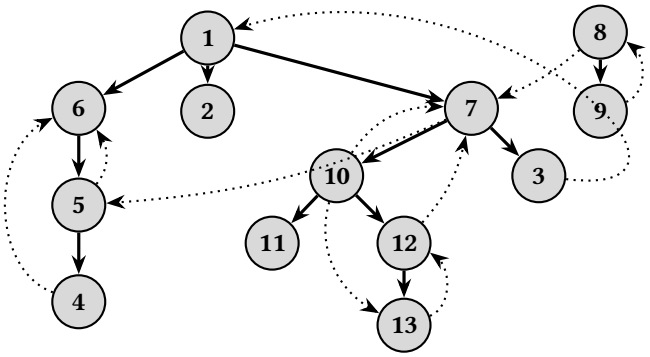



**Figura 5.5:** Esquema de uma busca rotula em 5.4

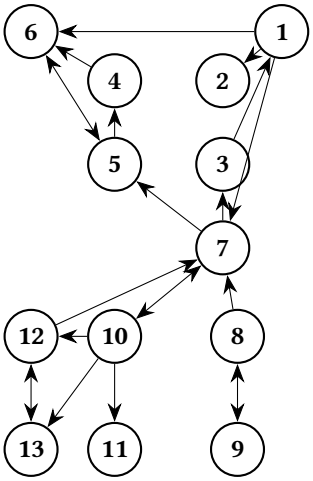



**Figura 5.4:** Exemplo de BP em grafo dirigido

Classifique os arcos do grafo dirigido após a busca em profundidade.

## 5.2 Grafos dirigidos acíclicos e ordenação topológica

*Nesta seção todos os grafos dirigidos são simples, isto é, sem laços.*

Em muitos problemas os arcos de um grafo dirigido representam relações de precedência: um arco $(u, v)$ indica que $u$ deve vir antes de $v$. É o caso de tarefas de um projeto em que algumas só podem começar depois que outras terminam, de disciplinas com pré-requisitos, ou de arquivos de um programa em que a compilação de um depende de outro. Nessas situações queremos uma sequência que respeite todas as precedências. Isso é impossível se as precedências formam um circuito: $u$ antes de $v$, $v$ antes de $w$ e $w$ antes de $u$. Nesta seção mostramos que essa é a única obstrução e como encontrar uma tal sequência em tempo linear.

**Definição 5.1: Caminho e circuito orientados**

Um ***caminho orientado*** em $G$ é uma sequência de vértices distintos $x_1, \dots, x_k$, com $k \geq 1$ e $(x_i, x_{i+1}) \in E(G)$ para todo $i \in \{1, \dots, k-1\}$.
Um ***circuito orientado*** é uma sequência $x_1, \dots, x_k, x_1$, em que $x_1, \dots, x_k$ é um caminho orientado com $k \geq 2$ e $(x_k, x_1) \in E(G)$. (Como não há laços, $k = 1$ não ocorre; para $k = 2$ o circuito usa os arcos $(x_1, x_2)$ e $(x_2, x_1)$.)

**Definição 5.2: grafo dirigido acíclico**

Um ***grafo dirigido acíclico*** (GDA, em inglês *DAG*) é um grafo dirigido que não contém circuito orientado.

**Exemplo 5.1**

Considere cinco disciplinas $a, b, c, d, e$ com os seguintes pré-requisitos: $a$ é pré-requisito de $b$ e de $c$; $b$ e $c$ são pré-requisitos de $d$; $c$ é pré-requisito de $e$. O grafo dirigido $G$ dado por $V(G) = \{a, b, c, d, e\}$ e

$$E(G) = \{(a, b), (a, c), (b, d), (c, d), (c, e)\}$$

modela essa situação e é acíclico (Figura 5.6). A sequência $a, b, c, d, e$ respeita os pré-requisitos, assim como $a, c, e, b, d$. Já a sequência $a, b, d, c, e$ não serve, pois $c$ é pré-requisito de $d$ mas vem depois dele.

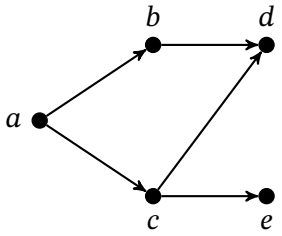



**Figura 5.6:** grafo dirigido do Exemplo 5.1.

**Definição 5.3: ordenação topológica**

Seja $G$ um grafo dirigido com $n$ vértices. Uma ***ordenação topológica*** de $G$ é uma ordenação $v_1, v_2, \ldots, v_n$ dos vértices de $G$, cada vértice aparecendo exatamente uma vez, tal que

$$(v_i, v_j) \in E(G) \implies i < j.$$

Escrevendo $u \prec v$ quando $u$ aparece antes de $v$ na ordenação, a condição é que $u \prec v$ para todo arco $(u, v)$ de $G$. O ***problema da ordenação topológica*** é: dado um grafo dirigido $G$, determinar uma ordenação topológica de $G$ ou concluir que não existe.

Se dispomos os vértices em linha, na ordem da ordenação, todos os arcos apontam para a direita, como no diagrama da Figura 5.8. As ordenações do Exemplo 5.1 são exatamente as sequências de disciplinas que respeitam os pré-requisitos.

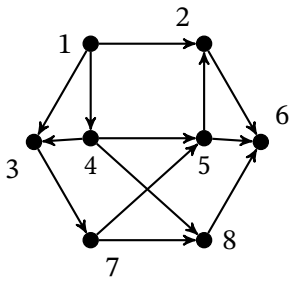



**Figura 5.7:** Um grafo dirigido e sua ordenação topológica mostrada na Figura 5.8.

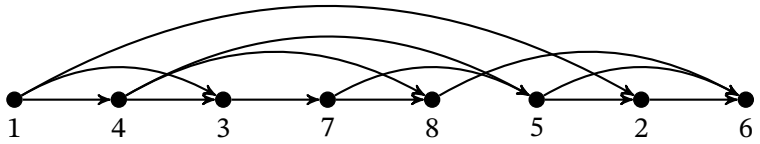



**Figura 5.8:** Uma ordenação topológica dos vértices do grafo da Figura 5.7: $1 \prec 4 \prec 3 \prec 7 \prec 8 \prec 5 \prec 2 \prec 6$.

Dizemos que um vértice $v$ é uma ***fonte*** se $\mathrm{d}^-(v) = 0$, isto é, nenhum arco chega em $v$, e que $v$ é um ***sumidouro*** se $\mathrm{d}^+(v) = 0$, isto é, nenhum arco sai de $v$. Em toda ordenação topológica o primeiro vértice é uma fonte e o último é um sumidouro: um arco que chegasse no primeiro, ou saísse do último, teria o outro extremo antes do primeiro ou depois do último vértice. Em um GDA sempre existem uma fonte e um sumidouro, e isso é o ponto de partida para provar a existência de ordenações topológicas.

**Proposição 5.2: Fontes e sumidouros**

Todo grafo dirigido acíclico com pelo menos um vértice tem uma fonte e um sumidouro.

*Demonstração.* Seja $G$ um GDA com pelo menos um vértice e tome um caminho orientado $P = x_1, x_2, \ldots, x_k$ com $k \geq 1$ máximo.

Suponha que existe um arco $(u, x_1) \in E(G)$. Como $G$ não tem laços, $u \neq x_1$. Se $u \notin \{x_1, \ldots, x_k\}$ então $u, x_1, \ldots, x_k$ é um caminho orientado com $k+1$ vértices, contradizendo a maximalidade de $P$. Se $u = x_i$ para algum $i \in \{2, \ldots, k\}$ então $x_1, \ldots, x_i, x_1$ é um circuito orientado em $G$, contradizendo que $G$ é acíclico. Logo nenhum arco chega em $x_1$, isto é, $\mathrm{d}^-(x_1) = 0$ e $x_1$ é uma fonte.

Analogamente, suponha que existe um arco $(x_k, w) \in E(G)$; en-

tão $w \neq x_k$, pois não há laços. Se $w \notin \{x_1, \ldots, x_k\}$, o caminho $x_1, \ldots, x_k, w$ contradiz a maximalidade de $P$; se $w = x_i$ com $i \in \{1, \ldots, k-1\}$, então $x_i, \ldots, x_k, x_i$ é um circuito orientado em $G$, uma contradição. Portanto $\mathrm{d}^+(x_k) = 0$ e $x_k$ é um sumidouro. □

**Teorema 5.1: Existência de ordenação topológica**

Um grafo dirigido $G$ admite uma ordenação topológica se e somente se $G$ é acíclico.

*Demonstração.* (⇒) Seja $v_1, \ldots, v_n$ uma ordenação topológica de $G$ e denote por $\mathrm{pos}(v)$ o índice de $v$ nessa ordenação. Suponha, por absurdo, que $C = x_1, \ldots, x_k, x_1$ é um circuito orientado em $G$. Os arcos $(x_i, x_{i+1})$ dão

$$\mathrm{pos}(x_1) < \mathrm{pos}(x_2) < \cdots < \mathrm{pos}(x_k),$$

e o arco $(x_k, x_1)$ dá $\mathrm{pos}(x_k) < \mathrm{pos}(x_1)$. Juntos, $\mathrm{pos}(x_1) < \mathrm{pos}(x_k) < \mathrm{pos}(x_1)$, o que é absurdo.

(⇐) Por indução em $n = |V(G)|$. Se $n = 1$ não há arcos, pois não há laços, e a única ordenação do vértice é uma ordenação topológica. Seja $n \geq 2$ e suponha o resultado válido para GDAs com $n-1$ vértices. Pela Proposição 5.2, $G$ tem uma fonte $s$. O grafo $G' \coloneqq G\big[V(G) \setminus \{s\}\big]$ tem $n-1$ vértices e é acíclico, pois todo circuito orientado de $G'$ é um circuito orientado de $G$. Pela hipótese de indução, $G'$ tem uma ordenação topológica $w_1, \ldots, w_{n-1}$. Afirmamos que $s, w_1, \ldots, w_{n-1}$ é uma ordenação topológica de $G$. De fato, os arcos de $G$ que não incidem em $s$ são arcos de $G'$ e respeitam a ordem de $w_1, \ldots, w_{n-1}$. Como $\mathrm{d}^-(s) = 0$, todo arco que incide em $s$ é da forma $(s, w_j)$, e $s$ vem antes de $w_j$. □

A demonstração sugere um algoritmo: enquanto restarem vértices, remover uma fonte do grafo restante e colocá-la na próxima posição. Se em algum momento o grafo restante, que é não-vazio, não tem fonte, então, pela Proposição 5.2, ele não é acíclico, e portanto contém um circuito orientado, que também é um circuito de $G$ (o restante é um subgrafo induzido de $G$). Assim o algoritmo decide se $G$ é acíclico e, em caso afirmativo, devolve uma ordenação topológica. O Exercício 85 pede para implementá-lo em tempo $O(|V| + |E|)$. Vejamos outro algoritmo, baseado na busca em profundidade rotulada, que também decide se $G$ é acíclico.

### 5.2.1 Ordenação topológica por busca em profundidade

A ideia é a seguinte. Empilhar $u$ corresponde a *começar* a explorar $u$, e desempilhar $u$ corresponde a *terminar* de explorar $u$. Enquanto $u$ está na pilha, a busca está "dentro" de $u$, e $u$ só é desempilhado depois que todos os caminhos que partem dele foram esgotados. Assim, se

$u \to v$, então $v$ faz parte do que $u$ precisa terminar de explorar, e portanto $v$ termina antes de $u$, isto é, sai$(v) <$ sai$(u)$. Listando os vértices em ordem decrescente de sai, cada $u$ aparece antes de todos os seus sucessores, que é exatamente o que uma ordenação topológica exige.

Para ver isso com mais cuidado, considere o instante em que a busca examina uma aresta $u \to v$. Nesse momento $u$ está na pilha e $v$ está em um de três estados:

- $v$ ainda não foi visitado: ele é empilhado por cima de $u$ e, como a pilha é LIFO, sai antes de $u$;
- $v$ já foi visitado e já saiu da pilha: então $v$ saiu antes de $u$, pois $u$ ainda está na pilha;
- $v$ já foi visitado e ainda está na pilha: então $v$ é um ancestral de $u$ na busca, ou seja, existe um caminho de $v$ até $u$. Junto com a aresta $u \to v$ isso formaria um ciclo, o que é impossível num DAG.

Portanto, nos dois casos possíveis, sai$(v) <$ sai$(u)$. A hipótese de $G$ ser acíclico é usada apenas no último caso. De fato, em um grafo com ciclo a aresta de retorno $u \to v$ satisfaz sai$(v) >$ sai$(u)$, e a ordem por sai decrescente viola essa aresta.

Em uma frase: *ao terminar $u$, todos os seus sucessores já terminaram, pois ou já estavam terminados, ou foram explorados dentro da chamada de $u$; um sucessor ainda em andamento seria ancestral de $u$, o que daria um ciclo.*

**Exemplo 5.2**

Executando OrdTop($G$) no grafo do Exemplo 5.1, com os vértices percorridos em ordem alfabética (em $V(G)$ e em cada $N^+(v)$), obtemos

| $v$ | $a$ | $b$ | $c$ | $d$ | $e$ |
|---|---|---|---|---|---|
| chega[$v$] | 1 | 2 | 6 | 3 | 7 |
| sai[$v$] | 10 | 5 | 9 | 4 | 8 |

e a lista ord no final é $a, c, e, b, d$, que lista os vértices em ordem decrescente de sai[ ] e é uma das ordenações do Exemplo 5.1. O arco $(c, d)$ é transversal: é examinado quando $d$ já terminou e, nesse caso, sai$[d] <$ sai$[c]$.

Para obter a lista sem ordenar, inserimos cada vértice no início de uma lista ord no instante em que ele termina. Como o último vértice a terminar é inserido por último, ele fica na frente: ao final, ord lista os vértices em ordem decrescente de sai[ ].

As variáveis cont, chega, sai, p, ord e circ são globais; o vetor p não é usado pelo algoritmo, apenas pela análise.

**Algoritmo 26:** VisitaOT($G, w$).

**Dado** : um grafo dirigido $G$ e um vértice $w$ com chega[$w$] = 0.
**Devolve** : visita $w$ e os vértices ainda não visitados alcançáveis a partir de $w$.

```
1  cont ← cont + 1;
2  chega[w] ← cont;
3  para cada u ∈ N⁺(w) faça
4      se chega[u] = 0 então
5          p[u] ← w;
6          VisitaOT(G, u);
7      senão
8          se sai[u] = 0 então circ ← verdadeiro;
9  cont ← cont + 1;
10 sai[w] ← cont;
11 insira w no início da lista ord;
```

**Algoritmo 27:** OrdTop($G$).

**Dado** : um grafo dirigido $G$.
**Devolve** : uma ordenação topológica de $G$, se $G$ é acíclico; caso contrário, a informação de que $G$ contém circuito orientado.

```
1  para cada v ∈ V(G) faça
2      chega[v] ← 0;
3      sai[v] ← 0;
4      p[v] ← nil;
5  cont ← 0;
6  ord ← lista vazia;
7  circ ← falso;
8  para cada v ∈ V(G) faça
9      se chega[v] = 0 então VisitaOT(G, v);
10 se circ então
11     retorne G contém circuito orientado
12 retorne ord
```

### Análise do algoritmo

Fixe um grafo dirigido $G$ e considere uma execução de OrdTop($G$). Pela estrutura recursiva do algoritmo valem as seguintes observações, que usaremos sem mais comentários.

(a) Para todo $v \in V(G)$ a chamada VisitaOT($G, v$) é feita exatamente uma vez. De fato, ela só é feita se chega[$v$] = 0 e seus dois primeiros comandos tornam chega[$v$] $\neq$ 0; além disso, ao

final do laço principal todo vértice tem chega $\neq 0$. Em particular, cada arco $(w, u)$ é examinado exatamente uma vez, no laço da chamada de $w$.

(b) O contador cont aumenta de uma unidade antes de cada atribuição a chega[ ] ou a sai[ ]. Logo esses $2|V(G)|$ valores são inteiros distintos e chega[$v$] < sai[$v$] para todo $v$.

(c) As chamadas recursivas são aninhadas: a chamada de $u$ feita na chamada de $w$ termina antes de $w$ prosseguir. Em cada instante, os vértices com chega $\neq 0$ e sai $= 0$ (chamadas *ativas*) são o vértice $z$ cuja chamada está em execução e seus ancestrais na floresta definida por p[ ].

(d) Se chega[$w$] < chega[$x$] < sai[$w$], então $x$ foi descoberto durante a chamada de $w$, portanto por uma chamada aninhada nela; logo $x$ é descendente próprio de $w$ e sai[$x$] < sai[$w$].

(e) Para todo arco $(w, u) \in E(G)$, ao final do laço da chamada de $w$ temos chega[$u$] $\neq 0$: ou já era diferente de zero ao examinar o arco, ou a chamada recursiva o tornou. Como sai[$w$] é atribuído depois do laço, (b) dá chega[$u$] < sai[$w$].

(As relações entre os intervalos [chega, sai] e ancestralidade são as da Proposição 3.2 e do Exercício 58.)

**Lema 5.1: Exame de um arco**

Considere o instante em que um arco $(w, u) \in E(G)$ é examinado no laço da linha 3, durante a chamada de $w$. Então vale exatamente uma das situações:

1. chega[$u$] $= 0$; então $u$ é visitado em seguida, com p[$u$] $= w$, e sai[$u$] < sai[$w$];
2. chega[$u$] $\neq 0$ e sai[$u$] $\neq 0$; então sai[$u$] < sai[$w$];
3. chega[$u$] $\neq 0$ e sai[$u$] $= 0$; então $u$ é ancestral próprio de $w$ (isto é, $(w, u)$ é um arco de retorno ascendente) e sai[$w$] < sai[$u$].

Reciprocamente, se $u$ é ancestral próprio de $w$, o arco $(w, u)$ cai na situação 3.

*Demonstração.* Os três casos são mutuamente exclusivos e esgotam as possibilidades, pois sai[$u$] $\neq 0$ implica chega[$u$] $\neq 0$ por (b).

*Caso 1.* O algoritmo faz p[$u$] $\leftarrow w$ e chama VisitaOT($G, u$), que termina, e atribui sai[$u$], antes de $w$ prosseguir. Como sai[$w$] ainda não foi atribuído, por (b) temos sai[$u$] < sai[$w$].

*Caso 2.* O valor sai[$u$] já foi atribuído, enquanto sai[$w$] ainda não, pois a chamada de $w$ está em execução. Por (b), sai[$u$] < sai[$w$].

*Caso 3.* A chamada de $u$ está ativa. No instante do exame do arco, a chamada em execução é a de $w$, logo, por (c), os vértices ativos são $w$ e seus ancestrais. Como $G$ não tem laços, $u \neq w$, portanto $u$ é ancestral próprio de $w$. Sendo a chamada de $w$ aninhada na de $u$, ela termina

antes, isto é, $\text{sai}[w] < \text{sai}[u]$.

Reciprocamente, se $u$ é ancestral próprio de $w$ então, durante a chamada de $w$, a chamada de $u$ está ativa por (c), o que é a situação 3. □

**Proposição 5.3: Circuitos e retornos ascendentes**

Em uma execução de OrdTop($G$) ocorre algum arco de retorno ascendente se e somente se $G$ contém circuito orientado.

*Demonstração.* (⇒) Seja $(w, u)$ um arco de retorno ascendente, isto é, $u$ é ancestral próprio de $w$. Então existe $t \geq 1$ com $\text{p}^{(t)}[w] = u$ e, definindo $y_j := \text{p}^{(t-j)}[w]$ para $0 \leq j \leq t$, temos $y_0 = u$, $y_t = w$ e $\text{p}[y_j] = y_{j-1}$ para $1 \leq j \leq t$. Como $\text{p}[y_j]$ só é atribuído ao examinar o arco $(\text{p}[y_j], y_j)$, temos $(y_{j-1}, y_j) \in E(G)$. Os vértices $y_0, \ldots, y_t$ são distintos, pois $\text{chega}[\text{p}[v]] < \text{chega}[v]$ para todo $v$ (o pai é descoberto antes do filho) e, com o arco $(w, u) = (y_t, y_0)$, formam o circuito orientado $y_0, y_1, \ldots, y_t, y_0$.

(⇐) Seja $C = x_1, \ldots, x_k, x_1$ um circuito orientado em $G$. Trocando a numeração por uma rotação cíclica, podemos supor que $\text{chega}[x_1]$ é o menor valor de chega entre os vértices de $C$. Mostramos por indução em $i$, para $i = 2, \ldots, k$, que

$$\text{chega}[x_1] < \text{chega}[x_i] < \text{sai}[x_1].$$

Em particular, por (d), cada $x_i$ é descendente próprio de $x_1$ e $\text{sai}[x_i] < \text{sai}[x_1]$.

Para $i = 2$, por (e) aplicado a $(x_1, x_2)$ temos $\text{chega}[x_2] < \text{sai}[x_1]$, e $\text{chega}[x_2] > \text{chega}[x_1]$ pela escolha de $x_1$, pois $x_2 \neq x_1$. Isso mostra $Q_2$.

Suponha $Q_i$ para algum $i < k$. Por (d), $x_i$ é descendente próprio de $x_1$ e $\text{sai}[x_i] < \text{sai}[x_1]$; por (e) aplicado a $(x_i, x_{i+1})$,

$$\text{chega}[x_{i+1}] < \text{sai}[x_i] < \text{sai}[x_1],$$

e $\text{chega}[x_{i+1}] > \text{chega}[x_1]$ pela escolha de $x_1$. Isso mostra $Q_{i+1}$.

Assim $x_k$ é descendente próprio de $x_1$, isto é, $x_1$ é ancestral próprio de $x_k$, e $(x_k, x_1) \in E(G)$ é um arco de retorno ascendente. □

**Teorema 5.2: Corretude**

Ao final de OrdTop($G$) o indicador circ é verdadeiro se e somente se $G$ contém circuito orientado. Se $G$ é acíclico, a lista ord é uma ordenação topológica de $G$.

*Demonstração.* O indicador circ é atribuído na linha 8 precisamente quando um arco é examinado na situação 3 do Lema 5.1, isto é, quando

ocorre um arco de retorno ascendente. Pela Proposição 5.3, isso acontece se e somente se $G$ contém circuito orientado.

Suponha $G$ acíclico. Pelo que acabamos de provar, nenhum arco é examinado na situação 3, portanto todo arco $(w, u)$ cai na situação 1 ou 2 e satisfaz

$$\text{sai}[u] < \text{sai}[w].$$

Por (a), cada vértice é inserido em ord exatamente uma vez, na linha 11, logo após receber seu valor de sai[ ]. Como as inserções são no início da lista e os valores de sai[ ] são atribuídos em ordem crescente, ord contém os vértices em ordem decrescente de sai[ ]. Assim, para todo arco $(w, u)$, o vértice $w$ aparece antes de $u$ em ord, e ord é uma ordenação topológica de $G$. □

**Teorema 5.3: Complexidade**

Se $G$ é dado por listas de adjacência $N^+[\ ]$ e a inserção no início de ord custa $O(1)$, então OrdTop($G$) tem complexidade de tempo $O(|V| + |E|)$.

*Demonstração.* A inicialização custa $O(|V|)$ e o laço principal faz $|V|$ testes de custo $O(1)$, fora as chamadas. Por (a), cada vértice $w$ dispara uma única chamada de VisitaOT, que, descontadas as chamadas recursivas, executa $O(1)$ operações fora do laço da linha 3 (incluindo a inserção na linha 11) e $\mathrm{d}^+(w)$ iterações do laço, cada uma de custo $O(1)$. O custo total é, portanto,

$$O(|V|) + \sum_{w \in V} O\big(1 + \mathrm{d}^+(w)\big) = O(|V| + |E|),$$

pois $\sum_{w \in V} \mathrm{d}^+(w) = |E|$. □

Se $G$ é dado por sua matriz de adjacência, percorrer $N^+(w)$ custa $\Theta(|V|)$ para cada $w$ e o algoritmo tem complexidade $\Theta(|V|^2)$. A pilha de recursão tem profundidade até $|V|$, e o espaço total é $O(|V|)$ além da representação de $G$.

## 5.3 Componentes fortemente conexas

**Definição 5.4: componente fortemente conexa**

Uma ***componente fortemente conexa*** num grafo dirigido $G$ é um subconjunto de vértices $C \subseteq V(G)$ maximal com respeito a seguinte propriedade: para quaisquer $u, v \in C$ existem um $(u \to v)$-caminho orientado e um $(v \to u)$-caminho orientado, ambos contidos em $G[C]$.

Equivalentemente, a relação $\leftrightsquigarrow$ sobre $V$ dada por $u \leftrightsquigarrow$ se e somente se existem $(u \to v)$-caminho e $(v \to u)$-caminho orientados em $G$

é de equivalência. As classes de equivalência são as componentes fortemente conexas do grafo dirigido $G$.

O diagrama da Figura 5.9 abaixo mostra um grafo dirigido, cada um dos quatro círculos identifica uma componente fortemente conexa do grafo.

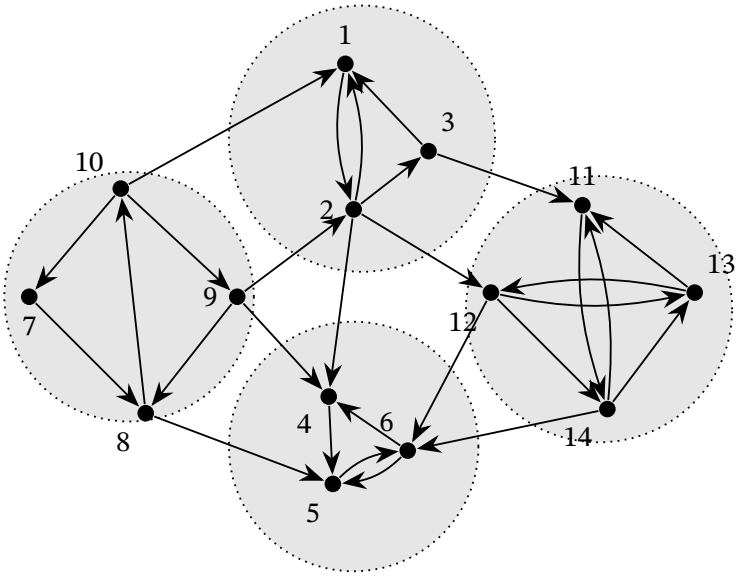



**Figura 5.9:** Um grafo dirigido e suas componentes fortemente conexas, destacadas em cinza.

Observemos que se um caminho orientado deixa uma componente fortemente conexa $C$, então esse caminho não pode retornar a $C$ pois, caso contrário, todos os vértices de $C$ deveriam pertencer a $C$. Em particular, cale o seguinte resultado.

**Proposição 5.4**

Sejam $C$ e $C'$ componentes fortemente conexas distintas de um grafo dirigido simples $G$. Então todas as arestas entre $C$ e $C'$ têm o mesmo sentido: ou todas vão de $C$ para $C'$, ou todas vão de $C'$ para $C$.

*Demonstração.* Suponha, por absurdo, que existam arestas $u \to v$ e $y \to w$ com $u, w \in C$ e $v, y \in C'$.

Como $C$ é fortemente conexa, existe um caminho de $w$ até $u$ dentro de $C$. Como $C'$ é fortemente conexa, existe um caminho de $v$ até $y$ dentro de $C'$. Concatenando, obtemos um caminho de $v$ até $u$

$$v \rightsquigarrow y \to w \rightsquigarrow u.$$

Junto com a aresta $u \to v$, isso mostra que $u$ e $v$ são alcançáveis um a partir do outro, isto é, $u$ e $v$ pertencem à mesma componente fortemente conexa. Mas $u \in C$, $v \in C'$ e $C \neq C'$, o que é uma contradição. □

Em vista disso que acabamos de provar, fica bem definido o grafo das componentes que passamos a definir em seguida.

**Definição 5.5: Grafo $G^*$ das cfc**

O ***grafo das componentes*** de $G$, denotado por $G^*$ é o grafo orientado com um vértice para cada componente fortemente conexa de $G$ e $(c_1, c_2)$ é um arco em $G^*$ se, e somente se, existe um arco que sai de um vértice de $c_1$ e chega num vértice de $c_2$ (veja um exemplo na Figura 5.10(a) abaixo).

**Proposição 5.5: $G^*$ é G.D.A.**

Seja $G$ um grafo dirigido e seja $G^*$ o grafo das componentes fortemente conexas de $G$. Então $G^*$ é acíclico.

*Demonstração.* Suponha, por absurdo, que $G^*$ tenha um ciclo dirigido

$$C_1 \to C_2 \to \cdots \to C_k \to C_1,$$

com $k \geq 2$ e as componentes $C_1, \ldots, C_k$ duas a duas distintas (basta tomar um ciclo de comprimento mínimo; $k \geq 2$ pois $G^*$ não tem laços).

Para cada $i$, pela definição das arestas de $G^{\text{SCC}}$, existe em $G$ uma aresta $u_i \to v_{i+1}$ com $u_i \in C_i$ e $v_{i+1} \in C_{i+1}$ (os índices são tomados módulo $k$). Como $C_i$ é fortemente conexa, existe um caminho em $G$ de $v_i$ até $u_i$ dentro de $C_i$. Assim, para cada $i$ há um caminho de $v_i$ até $v_{i+1}$:

$$v_i \rightsquigarrow u_i \to v_{i+1}.$$

Concatenando esses caminhos para $i = 1, \ldots, k$, obtemos um caminho de $v_1$ até $v_{k+1} = v_1$ passando por $v_2, \ldots, v_k$. Em particular, existe um caminho de $v_2$ até $v_1$ (seguindo $v_2 \rightsquigarrow v_3 \rightsquigarrow \cdots \rightsquigarrow v_k \rightsquigarrow v_1$) e um caminho de $v_1$ até $v_2$. Logo $v_1$ e $v_2$ são mutuamente alcançáveis e pertencem à mesma componente fortemente conexa. Mas $v_1 \in C_1$, $v_2 \in C_2$ e $C_1 \neq C_2$, uma contradição. □

**Corolário 5.1**

Os vértices de $G^*$ admitem uma ordenação topológica. □

Na figura abaixo mostramos (a) o grafo das componentes fortemente conexas do grafo dirigido mostrado na Figura 5.9 e também (b) uma ordenação topológica dos vértices do grafo das componentes.

Agora, vamos descrever um algoritmo que, dado um grafo dirigido $G$, determina as componentes fortemente conexas de $G$. Para isso, definimos, ainda, o ***grafo transposto***: denotado por $G^\intercal$ é o grafo dirigido obtido invertendo-se o sentido dos arcos em $E(G)$: $G^\intercal = \big(V(G), \{(u, v) \in V \times V : (v, u) \in E(G)\}\big)$.

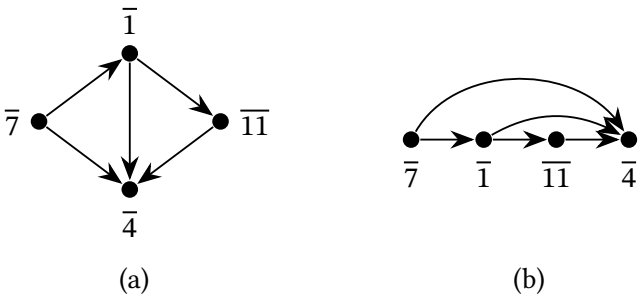



**Figura 5.10:** O grafo das componentes fortemente conexas (a) e uma ordenação topológica (b).

**Observação 5.1: Transposição não altera componentes**

Os grafos $G$ e $G^\intercal$ têm as mesmas componentes fortemente conexas. De fato, se $u$ e $v$ estão no mesmo componente $C$ de $G$ então o $(u \to v)$-caminho em $G$ com todos os vértices em $C$ é um $(v \to u)$-caminho em $G^\intercal$ com todos os vértices em $C$ e, analogamente, o $(v \to u)$-caminho em $G$ é um $(u \to v)$--caminho em $G^\intercal$, portanto, $u$ e $v$ estão no componente $C$ de $G^\intercal$.

**Observação 5.2: Transposição inverte $G^*$**

Para componentes distintas $C$ e $C'$, existe aresta $C \to C'$ em $G^*$ se e somente se existe aresta $C' \to C$ em $(G^\intercal)^*$. Isto é, $(G^\intercal)^* = (G^*)^\intercal$. De fato, uma aresta $u \to v$ de $G$ com $u \in C$ e $v \in C'$ corresponde à aresta $v \to u$ de $G^\intercal$, de $C'$ para $C$, e vice-versa.

### Intuição

Se contraímos cada componente fortemente conexa de $G$ a um vértice, obtemos o G.D.A. $G^*$. Uma componente *fonte* de $G^*$ é como uma "ilha de partida": dela se chega a outras componentes, mas nenhuma componente chega a ela. Se começássemos uma busca em um vértice de uma componente *sumidouro* de $G^*$, a busca visitaria exatamente essa componente e pararia, pois não há como sair dela. Bastaria então repetir: remover essa componente e buscar de novo.

O problema é que não sabemos de antemão quais vértices estão em componentes sumidouro. Dois truques resolvem isso.

- A primeira busca, em $G$, produz os valores de sai, e o vértice com maior sai está sempre em uma componente *fonte* de $G^*$: componentes "mais à montante", mais próximas das fontes, terminam depois.
- Ao transpor $G$, todas as arestas entre componentes se invertem, e uma componente fonte vira um *sumidouro*. A segunda busca, em $G^\intercal$, escolhendo raízes em ordem decrescente de sai, começa então sempre em um sumidouro do que ainda não foi visitado, e extrai uma componente de cada vez, sem "vazar" para as outras.

Em linhas gerais, a estratégia para computar as componentes fortemente conexas de um grafo dirigido $G$ é

**Algoritmo 28:** CFC($G$)

1 BP($G$): busca em profundidade rotulada em $G$ para determinar sai[$v$] para todo $v \in V(G)$;
2 Compute $G^\intercal$;
3 BP($G^\intercal$): busca em profundidade em $G^\intercal$ com o laço principal modificado para escolher os vértices em ordem decrescente dos valores de *sai*[ ] computado no passo 1;
4 Devolva cada árvore da segunda busca como uma componente fortemente conexa de $G$.

### Notação e fatos preliminares

No que segue sempre que nos referirmos aos valores de chega[ ] e sai[ ] trata-se do conteúdo desses vetores após a primeira busca em profundidade, salvo menção em contrário.

Seja $G = (V, E)$ um grafo dirigido. Para $C \subseteq V$ definimos

$$\text{chega}(C) := \min\{\text{chega}[v] : v \in C\}, \quad \text{sai}(C) := \max\{\text{sai}[v] : v \in C\}.$$

**Proposição 5.6: o primeiro vértice de $C$ tem o maior sai**

Seja $C$ uma componente fortemente conexa e seja $x \in C$ o vértice com $\text{chega}(x) = \text{chega}(C)$. Então $\text{sai}(x) = \text{sai}(C)$, e $\text{sai}(x) > \text{sai}(v)$ para todo $v \in C \setminus \{x\}$.

*Demonstração.* No instante chega[$x$], todo vértice de $C$ diferente de $x$ é não visitado, pois $x$ é o primeiro de $C$ a ser descoberto. Como $C$ é fortemente conexa, para cada $v \in C$ existe um caminho de $x$ a $v$ dentro de $C$, portanto formado só por vértices não visitados (além de $x$).

**Observação 5.3: Teorema do caminho branco**

Em uma busca em profundidade, um vértice $y$ é descendente de $x$ na floresta de busca se e somente se, no instante chega[$x$], existe um caminho de $x$ a $y$ cujos vértices são todos ainda não visitados.

Pelo fato acima, todo $v \in C \setminus \{x\}$ é descendente de $x$, e portanto termina antes de $x$: $\text{sai}[v] < \text{sai}[x]$. □

Os seguintes resultados ajudam a entender o funcionamento do algoritmo.

**Lema 5.2**

Sejam $C$ e $C'$ duas componentes fortemente conexas de $G$. Suponha que existe um arco $(u, v) \in E(G)$ tal que $u \in C$ e $v \in C'$. Então $\text{sai}(C) > \text{sai}(C')$.

Intuitivamente: se $C$ aponta para $C'$, então $C'$ é "mais ao sumidouro" e termina antes.

*Demonstração.* Como $G^*$ é acíclico, não existe caminho em $G$ de nenhum vértice de $C'$ a nenhum vértice de $C$. (Caso contrário, juntando-o à aresta de $C$ para $C'$ e a caminhos dentro das componentes, teríamos um ciclo passando por $C$ e $C'$ em $G^*$.) Consideramos dois casos.

*Caso 1:* $\text{chega}(C) < \text{chega}(C')$. Seja $x \in C$ com $\text{chega}(x) = \text{chega}(C)$. No instante $\text{chega}[x]$, todos os vértices de $C \cup C'$ diferentes de $x$ são brancos: os de $C$ porque $x$ é o primeiro de $C$ a ser descoberto, e os de $C'$ porque $\text{chega}(C') > \text{chega}[x]$. Seja $u \to v$ uma aresta de $G$ com $u \in C$ e $v \in C'$. Para qualquer $w \in C'$, existe um caminho

$$x \rightsquigarrow u \to v \rightsquigarrow w,$$

com o primeiro trecho dentro de $C$, o último dentro de $C'$, e todos os vértices (exceto $x$) não visitados. Portanto, pelo teorema do caminho branco (Obs. 5.3) todo vértice de $C'$ é descendente de $x$, e portanto $\text{sai}[w] < \text{sai}[x]$ para todo $w \in C'$. Pela Proposição 5.6, $\text{sai}[x] = \text{sai}(C)$, logo $\text{sai}(C') < \text{sai}(C)$.

*Caso 2:* $\text{chega}(C') < \text{chega}(C)$. Seja $y \in C'$ com $\text{chega}[y] = \text{chega}(C')$. Pela Proposição 5.6, $\text{sai}(C') = \text{sai}[y]$. Todo descendente de $y$ é alcançável a partir de $y$; como nenhum vértice de $C$ é alcançável a partir de um vértice de $C'$, nenhum vértice de $C$ é descendente de $y$. Logo nenhum vértice de $C$ é descoberto durante o intervalo $[\text{chega}[y], \text{sai}[y]]$. Como $\text{chega}(C) > \text{chega}(C') = \text{chega}[y]$, segue que $\text{chega}(C) > \text{sai}[y] = \text{sai}(C')$. Portanto

$$\text{sai}(C) > \text{chega}(C) > \text{sai}(C').$$

Nos dois casos, $\text{sai}(C) > \text{sai}(C')$. □

**Observação 5.4**

Seja $\mathcal{U}$ um conjunto de componentes e seja $C \in \mathcal{U}$ com $\text{sai}(C)$ máximo em $\mathcal{U}$. Então não existe aresta $C' \to C$ em $G^*$ com $C' \in \mathcal{U}$, $C' \neq C$. Com efeito, pelo Lema 5.2 teríamos $\text{sai}(C') > \text{sai}(C)$, contrariando a maximalidade de $C$.

**Corolário 5.2**

Sejam $C$ e $C'$ duas componentes fortemente conexas de $G$. Suponha que existe um arco $(u, v) \in E(G^\intercal)$ tal que $u \in C$ e $v \in C'$. Então

$$\text{sai}(C) < \text{sai}(C').$$

*Demonstração.* Imediato da definição de $G^\intercal$ e do resultado acima pois $G$ e $G^\intercal$ têm as mesmas componentes fortemente conexas. □

Intuitivamente, ao transpor o grafo, a componente que era fonte (entre as não visitadas) passa a ser sumidouro, e a busca não consegue escapar dela.

### Análise e correção do algoritmo CFC

**Teorema 5.4: Corretude de CFC**

Cada árvore da floresta produzida pela segunda busca (em $G^\intercal$, com raízes escolhidas em ordem decrescente do valor sai calculado na primeira busca) é exatamente o conjunto de vértices de uma componente fortemente conexa de $G$, e toda componente aparece como uma árvore.

*Demonstração.* Sejam $r_1, r_2, \ldots$ as raízes da segunda busca, na ordem em que são escolhidas. Mostramos, por indução no número de árvores já construídas, que vale o seguinte invariante:

> (I) *imediatamente antes da escolha de cada raiz, o conjunto $S$ dos vértices já visitados pela segunda busca é uma união de componentes fortemente conexas de $G$, e cada árvore construída até então é exatamente uma dessas componentes.*

Para a base, $S = \varnothing$. Para o passo, suponha que (I) vale antes da escolha de uma raiz $r$ e seja $C$ a componente fortemente conexa de $G$ que contém $r$. Seja $\mathcal{U}$ o conjunto das componentes contidas em $\overline{S}$; por (I), $\overline{S}$ é a união das componentes de $\mathcal{U}$. Como $r \notin S$, temos $C \in \mathcal{U}$, ou seja, nenhum vértice de $C$ foi visitado até este instante. Além disso, os vértices ainda não visitados pela segunda busca são exatamente os de $\overline{S}$.

*Afirmação 1:* $\text{sai}(C) \geq \text{sai}(D)$ *para toda* $D \in \mathcal{U}$. Pela regra de escolha, $r$ é o vértice de $\overline{S}$ com maior valor de sai. Como $r \in C \subseteq \overline{S}$, temos $\text{sai}(C) = \text{sai}[r]$. Por outro lado, para cada $D \in \mathcal{U}$, o valor $\text{sai}(D)$ é atingido por um vértice de $D \subseteq \overline{S}$, logo $\text{sai}(D) \leq \text{sai}[r] = \text{sai}(C)$.

*Afirmação 2: nenhum arco de* $G^\intercal$ *vai de* $C$ *a um vértice de* $\overline{S}$ *fora de* $C$. Suponha que $(u, v) \in E(G^\intercal)$ com $u \in C$, $v \in \overline{S}$ e $v \notin C$, e seja $D$ a componente de $v$. Então $D \in \mathcal{U} \setminus \{C\}$ e, pelo Corolário 5.2, $\text{sai}(C) < \text{sai}(D)$, o que contradiz a Afirmação 1.

*Afirmação 3: o conjunto de vértices da árvore $T_r$ da segunda busca com raiz $r$ é exatamente $C$.* No instante em que $r$ é descoberto, os vértices ainda não visitados são exatamente os de $\overline{S}$. Assim:

- ($\supseteq$) Seja $v \in C$. Pela Observação 5.1, $C$ é fortemente conexa em $G^\intercal$, logo existe em $G^\intercal$ um caminho de $r$ a $v$ com todos os vértices em $C \subseteq \overline{S}$, isto é, todos ainda não visitados. Pelo teorema do caminho de vértices não visitados (Obs. 5.3), $v$ é descendente de $r$.
- ($\subseteq$) Seja $z$ um descendente de $r$. Pelo mesmo teorema, existe em $G^\intercal$ um caminho de $r$ a $z$ cujos vértices pertencem todos a $\overline{S}$. Se $z \notin C$, a primeira aresta desse caminho que sai de $C$ é um arco de $G^\intercal$ de $C$ a um vértice de $\overline{S}$ fora de $C$, o que contradiz a Afirmação 2. Logo $z \in C$.

Pela Afirmação 3, a árvore de $r$ é exatamente a componente $C$ e, portanto, ao final de sua construção, o conjunto de vértices visitados é $S \cup C$, uma união de componentes: o invariante (I) é preservado.

Como o laço principal só termina quando todos os vértices foram visitados, cada vértice pertence a alguma árvore, e cada árvore é a componente de sua raiz. Logo cada árvore é uma componente fortemente conexa de $G$ e toda componente aparece como uma (única) árvore. □

### Uma implementação

O Algoritmo 30 usa o vetor cfc[ ] para identificar, no final da execução, as componentes fortemente conexas de $G$. Os vetores chega, sai e cfc e o contador *cont* são variáveis globais. A busca em profundidade utilizada é o Algoritmo 29:

**Algoritmo 29:** BP($G, v, cc$)

**Dado** : um grafo dirigido $G$, um vértice $v$ e um rótulo $cc$.

**Devolve** : os vetores chega[ ], sai[ ] e cfc[ ] atualizados para os vértices alcançados a partir de $v$.

```
1 cfc[v] ← cc;
2 cont ← cont + 1;
3 chega[v] ← cont;
4 para cada u ∈ N⁺_G(v) faça
5     se chega[u] = 0 então BP(G, u, cc) ;
6 cont ← cont + 1;
7 sai[v] ← cont.
```

O parâmetro *cc* rotula os vértices visitados pela busca com o índice da componente fortemente conexa à qual pertencem: todos os vértices alcançados por uma chamada de BP a partir da raiz $v$ recebem cfc[·] = *cc*. Na primeira busca, feita em $G$, as componentes ainda não são

conhecidas e o rótulo não é usado; por isso passamos $cc = 0$, que significa "sem rótulo". Os rótulos $1, 2, \ldots$ só são atribuídos na segunda busca, feita em $G^\intercal$, e o algoritmo CFC fica

**Algoritmo 30:** CFC($G$)

**Dado** : um grafo dirigido $G$.
**Devolve** : vetor cfc[ ] das componentes fortemente conexas.

```
1  cont ← 0;
2  para cada k ∈ V(G) faça
3  |   sai[k] ← 0;
4  |   chega[k] ← 0;
5  para cada k ∈ V(G) faça
6  |   se chega[k] = 0 então BP(G, k, 0) ;
7  L ← lista ordenada de V(G) por ordem decrescente
     de sai[ ];
8  compute G^T;
9  cont ← 0;
10 c ← 0;
11 para cada k ∈ V(G) faça
12 |   sai[k] ← 0;
13 |   chega[k] ← 0;
14 para cada k ∈ L faça
15 |   se chega[k] = 0 então
16 |   |   c ← c + 1;
17 |   |   BP(G^T, k, c);
18 retorne cfc
```

### Complexidade

A primeira busca custa $O(|V| + |E|)$, a construção de $G^\intercal$ custa $O(|V| + |E|)$, a ordenação (*counting sort*) e a segunda busca custam $O(|V|+|E|)$, desde que os vértices sejam percorridos na ordem decrescente de sai (basta guardar os vértices em uma lista, na ordem de término, durante a primeira busca, e percorrê-la ao contrário). O custo total é $O(|V| + |E|)$.

## Exercícios

*Exercício* 84 (***Fecho transitivo***). Se $D = (V, E)$ é um grafo dirigido então o fecho transitivo de $D$ é o grafo $D^* = (V, E^*)$ onde

$$E^* = \{(i, j) \in V \times V : \text{ existe um } (i \to j)\text{-caminho em } D\}.$$

Escreva um algoritmo de tempo $O(|V|^3)$ para determinar o fecho transitivo de um grafo dirigido.

*Exercício* 85. Escreva um algoritmo que determina uma ordenação topológica removendo repetidamente uma fonte, como na demonstração do Teorema 5.1. Mantenha um vetor com os graus de entrada $d^-[\,]$ e uma fila com as fontes atuais para obter tempo $O(|V| + |E|)$. Como decidir, ao fim, se $G$ é acíclico?

*Exercício* 86. Quantas ordenações topológicas tem o grafo do Exemplo 5.1?

*Exercício* 87. Prove que um grafo dirigido acíclico com $n$ vértices tem no máximo $\binom{n}{2}$ arcos e que existe um GDA com exatamente $\binom{n}{2}$ arcos.

*Exercício* 88. Prove que um grafo dirigido acíclico tem uma única ordenação topológica se e somente se contém um caminho orientado que passa por todos os vértices.

*Exercício* 89. Modifique o algoritmo OrdTop para que, quando $G$ contém circuito orientado, ele devolva um circuito orientado de $G$. (Dica: use o vetor p[ ] e a demonstração da Proposição 5.3.)

*Exercício* 90. Mostre que listar os vértices em ordem *crescente* de chega[ ] não dá, em geral, uma ordenação topológica de um GDA. (Dica: considere o grafo com $V = \{1, 2, 3\}$ e arcos $(1, 3)$ e $(2, 3)$.)

*Exercício* 91. Mostre que um grafo dirigido é acíclico se e somente se todas as suas componentes fortemente conexas (Seção 5.3) têm um único vértice.

*Exercício* 92. Se ao invés de usarmos uma busca em profundidade em $G^\intercal$ por ordem decrescente de *sai*[ ] no algoritmo 30 usarmos uma busca em profundidade em $G$ por ordem crescente de *chega*[ ] o algoritmo determinaria as componentes fortemente conexas do grafo?

*Exercício* 93. Se ao invés de usarmos uma busca em profundidade em $G^\intercal$ por ordem decrescente de *sai*[ ] no algoritmo 30 usarmos uma busca em profundidade em $G$ por ordem crescente de *sai*[ ] o algoritmo determinaria as componentes fortemente conexas do grafo?

*Exercício* 94. Seja $G$ um grafo dirigido e $u$ e $w$ vértices de $G$.

1. Escrevemos $u \rightsquigarrow w$ se existe $(u \to w)$-caminho em $G$. Prove que $\rightsquigarrow$ é uma relação simétrica e transitiva em $V(G)$.
2. Escrevemos $u \leftrightsquigarrow w$ se existem $(u \to w)$- e $(w \to u)$-caminhos em $G$. Prove que $\leftrightsquigarrow$ é uma relação de equivalência em $V(G)$. Nesse caso, quem são as classes de equivalência?

*Exercício* 95. Escreva um algoritmo para determinar se um grafo dirigido $G$ é ***semiconexo***: para $u, v \in V(G)$ ou $u \rightsquigarrow v$ ou $v \rightsquigarrow u$. Determine a complexidade do algoritmo.

*Exercício* 96. Escreva um algoritmo de complexidade $O(|V|+|E|)$ para computar o grafo $G^*$ das componentes fortemente conexas de $G$.

*Exercício* 97. Escreva um algoritmo de complexidade $O(|V|+|E|)$ para determinar o grafo transposto de um grafo dado por uma lista de adjacências.